

Some algorithms, like quicksort, need to mutate data in place in order to work correctly or efficiently. Fortunately for us, we can always safely mutate data that's created locally. Any function can use side-effecting components internally and still present a pure external interface to its callers, and we should feel no shame taking advantage of this in our own programs. We may prefer purely functional components in our implementations for other reasons—they're easier to get right, can be assembled more easily from other pure functions, and so on—but in principle there's nothing wrong with building a pure function using local side effects in the implementation.

14.2 A data type to enforce scoping of side effects

The preceding section makes it clear that pure functions may have side effects with respect to data that's *locally scoped*. The quicksort function may mutate the array because it allocated that array, it's locally scoped, and it's not possible for any outside code to observe the mutation. If, on the other hand, quicksort somehow mutated its input list directly (as is common in mutable collection APIs), that side effect would be observable to all callers of quicksort.

There's nothing wrong with doing this sort of loose reasoning to determine the scoping of side effects, but it's sometimes desirable to *enforce* effect scoping using Scala's type system. The constituent parts of quicksort would have direct side effects if used on their own, and with the types we're using, we get no help from the compiler in controlling the scope of these side effects. Nor are we alerted if we accidentally leak side effects or mutable state to a broader scope than intended. In this section, we'll develop a data type that uses Scala's type system to enforce scoping of mutations.¹

Note that we could just work in IO, but that's really not appropriate for local mutable state. If quicksort returned IO[List[Int]], then it would be an IO action that's perfectly safe to run and would have no side effects, which isn't the case in general for arbitrary IO actions. We want to be able to distinguish between effects that are safe to run (like locally mutable state) and external effects like I/O. So a new data type is in order.

14.2.1 A little language for scoped mutation

The most natural approach is to make a little language for talking about mutable state. Writing and reading a state is something we can already do with the State[S, A] monad, which you'll recall is just a function of type S => (A, S) that takes an input state and produces a result and an output state. But when we're talking about mutating the state *in place*, we're not really passing it from one action to the next. What we'll pass instead is a kind of token marked with the type S. A function called with the token then has the authority to mutate data that's tagged with the same type S.

This new data type will employ Scala's type system to gain two static guarantees. We want our code to *not compile* if it violates these invariants:

¹ There's a cost in terms of efficiency and notational convenience, so think of this as another technique you have at your disposal, not something that must be employed every time you make use of local mutation.

- If we hold a reference to a mutable object, then nothing can observe us mutating it.
- A mutable object can never be observed outside of the scope in which it was created.

We relied on the first invariant for our implementation of `quicksort`—we mutated an array, but since no one else had a reference to that array, the mutation wasn’t observable outside our function definition. The second invariant is more subtle; it’s saying that we won’t leak references to any mutable state as long as that mutable state remains in scope. This invariant is important for some use cases; see the sidebar.

Another use case for typed scoping of mutation

Imagine writing a file I/O library. At the lowest level, the underlying OS file read operation might fill up a mutable buffer of type `Array[Byte]`, reusing the same array on every read instead of allocating a new buffer each time. In the interest of efficiency, it might be nice if the I/O library could simply return a “read-only” view of type `Seq[Byte]` that’s backed by this array, rather than defensively copying the bytes to a fresh data structure. But this isn’t quite safe—the caller may keep around this (supposedly) immutable sequence, and when we overwrite the underlying array on the next read, that caller will observe the data changing out from under it! To make the recycling of buffers safe, we need to restrict the scope of the `Seq[Byte]` view we give to callers and make sure that callers can’t retain references (directly or indirectly) to these mutable buffers when we begin the next read operation that clobbers the underlying `Array[Byte]`. See the chapter notes for chapter 15 for more discussion of this sort of use case.

We’ll call this new local-effects monad `ST`, which could stand for *state thread*, *state transition*, *state token*, or *state tag*. It’s different from the `State` monad in that its `run` method is protected, but otherwise its structure is exactly the same.

Listing 14.2 Our new `ST` data type

```
sealed trait ST[S,A] { self =>
  protected def run(s: S): (A,S)
  def map[B](f: A => B): ST[S,B] = new ST[S,B] {
    def run(s: S) = {
      val (a, s1) = self.run(s)
      (f(a), s1)
    }
  }
  def flatMap[B](f: A => ST[S,B]): ST[S,B] = new ST[S,B] {
    def run(s: S) = {
      val (a, s1) = self.run(s)
      f(a).run(s1)
    }
  }
}
```